tpcc_dbfs_vs_postgres_basic_comparison

June 19, 2026

1 TPCC Comparison: dbfs vs PostgreSQL

This notebook compares BenchBase TPCC runs collected under the same nominal settings. Throughput/latency sections and buffer-pool diagnostic sections may use different result labels so instrumentation does not distort the reported throughput:

- throughput comparison label: configured by `throughput_comparison_root`
- statistics comparison label: configured by `stats_comparison_root`
- artifact selection: latest BenchBase artifact prefix under each engine directory
- throughput result directories contain dbfs and PostgreSQL artifacts
- statistics result directory supplies dbfs buffer-pool logs and events

The notebook reads summary, metrics, params, samples, results, raw traces, per-procedure result windows, and optional query-trace CSVs from those directories to build comparison tables and performance plots. If only one engine directory contains a completed run, the notebook displays that engine by itself.

[1]:
```
from pathlib import Path
import json
import re

import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import display

plt.style.use('ggplot')
pd.set_option('display.float_format', lambda value: f'{value:,.3f}')

throughput_comparison_root = Path(
    '/Users/yohasebe/Documents/dbfs/benchmarking/results/20260619-072546/tpcc')
stats_comparison_root = Path(
    '/Users/yohasebe/Documents/dbfs/benchmarking/results/20260619-074745/tpcc')

throughput_dbfs_result_dir = throughput_comparison_root / 'dbfs'
stats_dbfs_result_dir = stats_comparison_root / 'dbfs'

# Query traces describe the same uninstrumented run used for throughput.
query_trace_path = throughput_dbfs_result_dir / 'dbfs_query_trace.csv'
```

```python
if query_trace_path.exists():
    print(f"Loading query trace from: {query_trace_path}")
    query_trace_df = pd.read_csv(query_trace_path)

    # total_elapsed_us
    query_trace_sorted = query_trace_df.sort_values('total_elapsed_us',
                                                    ascending=False)

    # us ms
    display_df = query_trace_sorted.copy()
    display_df['total_elapsed_ms'] = display_df['total_elapsed_us'] / 1000
    display_df['mean_elapsed_ms'] = display_df['mean_elapsed_us'] / 1000
    display_df['max_elapsed_ms'] = display_df['max_elapsed_us'] / 1000

    display(display_df[[
        'kind', 'shape_key', 'executions', 'total_elapsed_ms',
        'mean_elapsed_ms', 'max_elapsed_ms', 'failures', 'last_error_type'
    ]])
else:
    print(f"No dbfs_query_trace.csv found under {throughput_dbfs_result_dir}")
```

```
Loading query trace from: /Users/yohasebe/Documents/dbfs/benchmarking/results/20
260619-072546/tpcc/dbfs/dbfs_query_trace.csv
```

	kind	shape_key
0	QUERY	SELECT I_PRICE, I_NAME , I_DATA FROM item WHER…
1	UPDATE	UPDATE stock SET S_QUANTITY = ? , S_YTD = S_YT…
2	QUERY	SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0…
3	UPDATE	UPDATE customer SET C_BALANCE = ?, C_YTD_PAYME…
4	UPDATE	DELETE FROM new_order WHERE NO_O_ID = ? AND NO…
5	QUERY	SELECT SUM(OL_AMOUNT) AS OL_TOTAL FROM order_l…
6	UPDATE	UPDATE warehouse SET W_YTD = W_YTD + ? WHERE W…
7	QUERY	SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_…
8	BATCH_UPDATE	INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W…
9	QUERY	SELECT C_DISCOUNT, C_LAST, C_CREDIT FROM custo…
10	QUERY	SELECT W_TAX FROM warehouse WHERE W_ID = ?
11	UPDATE	UPDATE district SET D_YTD = D_YTD + ? WHERE D_…
12	UPDATE	INSERT INTO history (H_C_D_ID, H_C_W_ID, H_C_I…
13	QUERY	SELECT D_NEXT_O_ID, D_TAX FROM district WHERE …
14	UPDATE	UPDATE order_line SET OL_DELIVERY_D = ? WHERE …
15	QUERY	SELECT NO_O_ID FROM new_order WHERE NO_D_ID = …
16	UPDATE	UPDATE district SET D_NEXT_O_ID = D_NEXT_O_ID …
17	UPDATE	INSERT INTO new_order (NO_O_ID, NO_D_ID, NO_W_…
18	UPDATE	UPDATE oorder SET O_CARRIER_ID = ? WHERE O_ID …
19	QUERY	SELECT D_STREET_1, D_STREET_2, D_CITY, D_STATE…
20	QUERY	SELECT W_STREET_1, W_STREET_2, W_CITY, W_STATE…
21	UPDATE	INSERT INTO oorder (O_ID, O_D_ID, O_W_ID, O_C_…
22	QUERY	SELECT O_C_ID FROM oorder WHERE O_ID = ? AND O…

23	QUERY	SELECT C_FIRST, C_MIDDLE, C_LAST, C_STREET_1, …		
24	QUERY	SELECT C_DATA FROM customer WHERE C_W_ID = ? A…		
25	QUERY	SELECT O_ID, O_CARRIER_ID, O_ENTRY_D FROM oord…		
26	QUERY	SELECT OL_I_ID, OL_SUPPLY_W_ID, OL_QUANTITY, O…		
27	UPDATE	UPDATE customer SET C_BALANCE = C_BALANCE + ?,…		
28	QUERY	SELECT COUNT(DISTINCT (S_I_ID)) AS STOCK_COUNT…		
29	UPDATE	UPDATE customer SET C_BALANCE = ?, C_YTD_PAYME…		
30	QUERY	SELECT D_NEXT_O_ID FROM district WHERE D_W_ID …		

	executions	total_elapsed_ms	mean_elapsed_ms	max_elapsed_ms	failures	\
0	2091	12,225.844	5.846	78.812	0	
1	2057	11,766.430	5.720	48.002	0	
2	2088	10,396.026	4.978	47.460	0	
3	195	3,395.480	17.412	45.710	0	
4	140	1,767.527	12.625	44.526	0	
5	140	1,745.064	12.464	44.523	0	
6	216	1,530.418	7.085	531.449	0	
7	140	1,369.694	9.783	157.798	0	
8	194	1,369.161	7.057	51.975	0	
9	197	1,027.769	5.217	46.107	0	
10	197	1,007.107	5.112	45.428	0	
11	216	902.752	4.179	45.860	0	
12	216	884.746	4.096	44.447	0	
13	197	791.908	4.019	42.695	0	
14	140	774.579	5.532	44.198	0	
15	140	731.312	5.223	45.844	0	
16	197	676.770	3.435	44.496	0	
17	197	676.262	3.432	43.617	0	
18	140	666.381	4.759	45.028	0	
19	216	657.867	3.045	45.621	0	
20	216	657.091	3.042	42.069	0	
21	197	530.443	2.692	45.516	0	
22	140	469.885	3.356	42.218	0	
23	90	439.527	4.883	42.224	0	
24	21	374.006	17.809	42.068	0	
25	14	364.054	26.003	159.626	0	
26	14	352.381	25.170	46.850	0	
27	140	289.185	2.065	43.339	0	
28	21	212.183	10.103	49.175	0	
29	21	128.514	6.119	40.734	0	
30	21	88.932	4.234	40.985	0	

	last_error_type
0	NaN
1	NaN
2	NaN
3	NaN
4	NaN

```
5               NaN
6               NaN
7               NaN
8               NaN
9               NaN
10              NaN
11              NaN
12              NaN
13              NaN
14              NaN
15              NaN
16              NaN
17              NaN
18              NaN
19              NaN
20              NaN
21              NaN
22              NaN
23              NaN
24              NaN
25              NaN
26              NaN
27              NaN
28              NaN
29              NaN
30              NaN
```

[2]:
```
run_configs = {
    'dbfs': {
        'label': 'tpcc_2026-05-22_06-34-29',
        'display_name': 'dbfs',
        'result_dir': throughput_comparison_root / 'dbfs',
    },
    'postgres': {
        'label': 'tpcc_2026-05-22_06-34-29',
        'display_name': 'PostgreSQL',
        'result_dir': throughput_comparison_root / 'postgres',
    },
}

configured_ratios = pd.Series(
    {
        'NewOrder': 45.0,
        'Payment': 43.0,
        'OrderStatus': 4.0,
        'Delivery': 4.0,
        'StockLevel': 4.0,
```

```
    },
    name='Configured Ratio (%)')

required_suffixes = {
    'summary': '.summary.json',
    'metrics': '.metrics.json',
    'params': '.params.json',
    'results': '.results.csv',
    'samples': '.samples.csv',
    'raw': '.raw.csv',
}

query_trace_filenames = {
    'dbfs': 'dbfs_query_trace.csv',
    'postgres': 'postgres_query_trace.csv',
}
procedure_suffix = '.results.'
```

[3]:
```
def load_terminal_log(result_dir):
    for filename in [
            'benchbase.log', 'benchbase_setup.log', 'benchbase_execute.log'
    ]:
        path = result_dir / filename
        if path.exists():
            return path.read_text(encoding='utf-8')
    return ''


run_configs = {
    engine: config
    for engine, config in run_configs.items()
    if (config['result_dir'].is_dir()
        and any(config['result_dir'].glob(f"*{required_suffixes['summary']}")))
}
if not run_configs:
    raise FileNotFoundError(
        f'No dbfs or postgres results found under {throughput_comparison_root}'
    )

missing_engines = sorted({'dbfs', 'postgres'} - set(run_configs))
if missing_engines:
    print('Skipping engines without results: ' + ', '.join(missing_engines))

for config in run_configs.values():
    config['terminal_log'] = load_terminal_log(config['result_dir'])

field_patterns = {
```

```
    'workload': re.compile(r'Benchmark:\s+(\w+)'),
    'dbms_type': re.compile(r'Type:\s+(\w+)'),
    'driver': re.compile(r'Driver:\s+(.+)'),
    'url': re.compile(r'URL:\s+(.+)'),
    'isolation': re.compile(r'Isolation:\s+(\S+)'),
    'loader_threads': re.compile(r'Loader Threads:\s+(\d+)'),
    'scale_factor': re.compile(r'Scale Factor:\s+([\d.]+)'),
    'terminals': re.compile(r'Terminals:\s+(\d+)'),
}
phase_pattern = re.compile(
    r'PHASE START :: .*\[Time=(\d+)\] \[WarmupTime=(\d+)\] \[Rate=([\d.]+)\].
 ↪*\[Ratios=\[([^\]]+)\]\]'
)
rate_pattern = re.compile(
    r'measuredRequests=(\d+)\) = ([\d.]+) requests/sec \(throughput\), ([\d.]+)␣
 ↪requests/sec \(goodput\)'
)


def parse_terminal_log(log_text):
    parsed = {}
    for key, pattern in field_patterns.items():
        match = pattern.search(log_text)
        parsed[key] = match.group(1).strip() if match else None

    phase_match = phase_pattern.search(log_text)
    if phase_match:
        parsed['phase_time_seconds'] = int(phase_match.group(1))
        parsed['warmup_time_seconds'] = int(phase_match.group(2))
        parsed['target_rate'] = float(phase_match.group(3))
        parsed['configured_ratio_list'] = [
            float(value.strip()) for value in phase_match.group(4).split(',')
        ]

    rate_match = rate_pattern.search(log_text)
    if rate_match:
        parsed['terminal_measured_requests'] = int(rate_match.group(1))
        parsed['terminal_throughput'] = float(rate_match.group(2))
        parsed['terminal_goodput'] = float(rate_match.group(3))

    return parsed


parsed_configs = pd.DataFrame([{
    'engine': engine,
    **parse_terminal_log(config['terminal_log'])
} for engine, config in run_configs.items()])
```

```
parsed_configs
```

```
Skipping engines without results: postgres
```

[3]:
```
  engine workload dbms_type                          driver  \
0   dbfs     TPCC  POSTGRES  dev.yohaku.dbfs.jdbc.DbfsDriver

                                  url                 isolation loader_threads  \
0  jdbc:dbfs://dbfs:25432/benchbase  TRANSACTION_SERIALIZABLE              2

  scale_factor terminals
0          1.0         1
```

[4]:
```
def find_latest_artifact_prefix(result_dir):
    summary_suffix = required_suffixes['summary']
    matches = sorted(result_dir.glob(f'*{summary_suffix}'))
    if not matches:
        raise FileNotFoundError(
            f'No artifact ending with {summary_suffix} under {result_dir}')
    latest_summary = matches[-1]
    return latest_summary.name[:-len(summary_suffix)]


def find_artifact(result_dir, run_label, suffix):
    artifact_path = result_dir / f'{run_label}{suffix}'
    if not artifact_path.exists():
        raise FileNotFoundError(
            f'No artifact ending with {suffix} for {run_label} under␣
 ↪{result_dir}'
        )
    return artifact_path


def load_optional_query_trace(engine, result_dir):
    trace_path = result_dir / query_trace_filenames[engine]
    if not trace_path.exists():
        return trace_path, None
    return trace_path, pd.read_csv(trace_path)


def load_run_artifacts(engine, config):
    result_dir = config['result_dir']
    detected_label = find_latest_artifact_prefix(result_dir)
    artifacts = {
        name: find_artifact(result_dir, detected_label, suffix)
        for name, suffix in required_suffixes.items()
    }
    procedure_paths = sorted(
```

```
        result_dir.glob(f'{detected_label}{procedure_suffix}*.csv'))
    query_trace_path, query_trace_df = load_optional_query_trace(
        engine, result_dir)

    with artifacts['summary'].open() as handle:
        summary = json.load(handle)
    with artifacts['metrics'].open() as handle:
        metrics = json.load(handle)
    with artifacts['params'].open() as handle:
        params = json.load(handle)

    results_df = pd.read_csv(artifacts['results'])
    samples_df = pd.read_csv(artifacts['samples'])
    raw_df = pd.read_csv(artifacts['raw'])
    procedure_dfs = {
        path.stem.split('.results.', 1)[1]: pd.read_csv(path)
        for path in procedure_paths
    }

    return {
        'label': detected_label,
        'artifacts': artifacts,
        'summary': summary,
        'metrics': metrics,
        'params': params,
        'results_df': results_df,
        'samples_df': samples_df,
        'raw_df': raw_df,
        'procedure_dfs': procedure_dfs,
        'query_trace_path': query_trace_path,
        'query_trace_df': query_trace_df,
    }


runs = {}
validation_rows = []
for engine, config in run_configs.items():
    run_data = load_run_artifacts(engine, config)
    runs[engine] = {**config, **run_data}
    validation_rows.extend([
        {
            'engine': engine,
            'artifact': 'results.csv',
            'rows': len(run_data['results_df']),
            'columns': ', '.join(run_data['results_df'].columns),
        },
        {
```

```
            'engine': engine,
            'artifact': 'samples.csv',
            'rows': len(run_data['samples_df']),
            'columns': ', '.join(run_data['samples_df'].columns),
        },
        {
            'engine': engine,
            'artifact': 'raw.csv',
            'rows': len(run_data['raw_df']),
            'columns': ', '.join(run_data['raw_df'].columns),
        },
        {
            'engine': engine,
            'artifact': 'procedure_results',
            'rows': len(run_data['procedure_dfs']),
            'columns': ', '.join(sorted(run_data['procedure_dfs'].keys())),
        },
        {
            'engine':
            engine,
            'artifact':
            query_trace_filenames[engine],
            'rows':
            0 if run_data['query_trace_df'] is None else len(
                run_data['query_trace_df']),
            'columns':
            'missing (optional)' if run_data['query_trace_df'] is None else
            ', '.join(run_data['query_trace_df'].columns),
        },
    ])

validation_df = pd.DataFrame(validation_rows)
validation_df
```

[4]:

	engine	artifact	rows	columns
0	dbfs	results.csv	12	Time (seconds), Throughput (requests/second), …
1	dbfs	samples.csv	60	Time (seconds), Requests, Throughput (requests…
2	dbfs	raw.csv	461	Transaction Type Index, Transaction Name, Star…
3	dbfs	procedure_results	5	Delivery, NewOrder, OrderStatus, Payment, Stoc…
4	dbfs	dbfs_query_trace.csv	31	kind, shape_key, sample_sql, executions, total…

[5]:
```
summary_rows = []
for engine, run in runs.items():
    parsed = parse_terminal_log(run['terminal_log'])
    summary = run['summary']
    latency = summary['Latency Distribution']

    summary_rows.append({
        'Engine':
        run['display_name'],
        'Run Label':
        run['label'],
        'Measured Requests':
        summary['Measured Requests'],
        'Elapsed Seconds':
        summary['Elapsed Time (nanoseconds)'] / 1_000_000_000,
        'Throughput (req/s)':
        summary['Throughput (requests/second)'],
        'Goodput (req/s)':
        summary['Goodput (requests/second)'],
        'Requests per Minute':
        summary['Throughput (requests/second)'] * 60,
        'Throughput - Goodput Gap':
        summary['Throughput (requests/second)'] -
        summary['Goodput (requests/second)'],
        'Average Latency (ms)':
        latency['Average Latency (microseconds)'] / 1000,
        'Median Latency (ms)':
        latency['Median Latency (microseconds)'] / 1000,
        'P95 Latency (ms)':
        latency['95th Percentile Latency (microseconds)'] / 1000,
        'P99 Latency (ms)':
        latency['99th Percentile Latency (microseconds)'] / 1000,
        'Max Latency (ms)':
        latency['Maximum Latency (microseconds)'] / 1000,
        'Terminal Throughput (req/s)':
        parsed.get('terminal_throughput'),
        'Terminal Goodput (req/s)':
        parsed.get('terminal_goodput'),
    })

summary_comparison_df = pd.DataFrame(summary_rows).set_index('Engine')
summary_comparison_df
```

[5]:

Engine	Run Label	Measured Requests	Elapsed Seconds	\
dbfs	tpcc_2026-06-18_22-30-47	461	60.000	

```
        Throughput (req/s)  Goodput (req/s)  Requests per Minute  \
Engine
dbfs                 7.683            7.650              460.999

        Throughput - Goodput Gap  Average Latency (ms)  Median Latency (ms)  \
Engine
dbfs                       0.033               129.710               88.217

        P95 Latency (ms)  P99 Latency (ms)  Max Latency (ms)  \
Engine
dbfs             332.656           537.337           922.864

       Terminal Throughput (req/s) Terminal Goodput (req/s)
Engine
dbfs                          None                       None
```

[6]:
```
transaction_names = [
    'NewOrder', 'Payment', 'OrderStatus', 'Delivery', 'StockLevel'
]
transaction_pattern = re.compile(r'procedures\.(\w+)/\d+\s+\[\s*(\d+)\]')


def extract_histogram_counts(log_text, section_name):
    section_pattern = re.compile(
        rf'{section_name}:\n(.*?)(?:\n\w[\w ]+:|\Z)',
        re.DOTALL,
    )
    match = section_pattern.search(log_text)
    if not match:
        return {name: 0 for name in transaction_names}

    block = match.group(1).strip()
    if '<EMPTY>' in block:
        return {name: 0 for name in transaction_names}

    counts = {name: 0 for name in transaction_names}
    for name, count in transaction_pattern.findall(block):
        counts[name] = int(count)
    return counts


mix_rows = []
for engine, run in runs.items():
    completed = extract_histogram_counts(run['terminal_log'],
                                         'Completed Transactions')
    aborted = extract_histogram_counts(run['terminal_log'],
                                       'Aborted Transactions')
```

```
    completed_total = sum(completed.values())
    aborted_total = sum(aborted.values())

    for name in transaction_names:
        mix_rows.append({
            'Engine':
            run['display_name'],
            'Transaction':
            name,
            'Completed':
            completed[name],
            'Aborted':
            aborted[name],
            'Observed Mix (%)': (completed[name] / completed_total *
                                 100) if completed_total else 0.0,
            'Abort Rate (%)':
            (aborted[name] / (completed[name] + aborted[name]) * 100) if
            (completed[name] + aborted[name]) else 0.0,
        })

transaction_mix_df = pd.DataFrame(mix_rows)
transaction_mix_df
```

[6]:

	Engine	Transaction	Completed	Aborted	Observed Mix (%)	Abort Rate (%)
0	dbfs	NewOrder	0	0	0.000	0.000
1	dbfs	Payment	0	0	0.000	0.000
2	dbfs	OrderStatus	0	0	0.000	0.000
3	dbfs	Delivery	0	0	0.000	0.000
4	dbfs	StockLevel	0	0	0.000	0.000

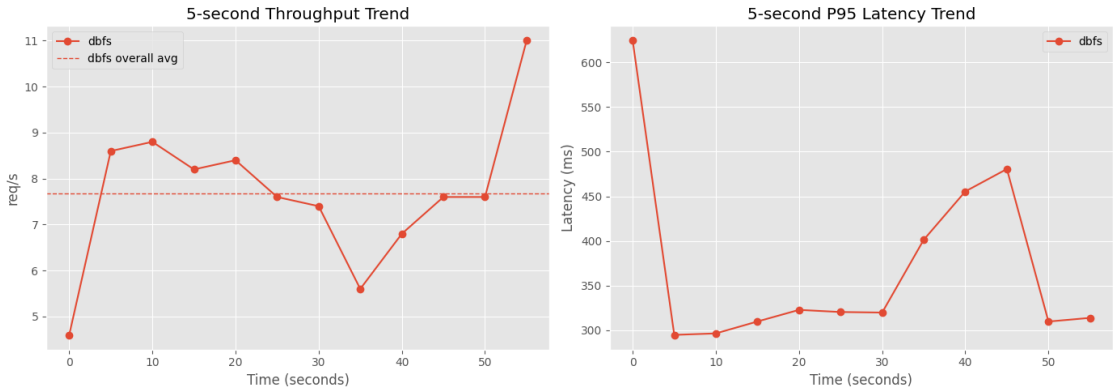
[7]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for engine, run in runs.items():
    results_df = run['results_df']
    label = run['display_name']
    axes[0].plot(
        results_df['Time (seconds)'],
        results_df['Throughput (requests/second)'],
        marker='o',
        label=label,
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['95th Percentile Latency (millisecond)'],
        marker='o',
        label=label,
    )
```

```
average_line_styles = {'dbfs': '--', 'postgres': ':'}
for engine, run in runs.items():
    display_name = run['display_name']
    axes[0].axhline(
        summary_comparison_df.loc[display_name, 'Throughput (req/s)'],
        linestyle=average_line_styles.get(engine, '--'),
        linewidth=1,
        label=f'{display_name} overall avg',
    )
axes[0].set_title('5-second Throughput Trend')
axes[0].set_xlabel('Time (seconds)')
axes[0].set_ylabel('req/s')
axes[0].legend()

axes[1].set_title('5-second P95 Latency Trend')
axes[1].set_xlabel('Time (seconds)')
axes[1].set_ylabel('Latency (ms)')
axes[1].legend()

fig.tight_layout()
plt.show()
```



[8]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for engine, run in runs.items():
    samples_df = run['samples_df'].copy()
    results_df = run['results_df']
    label = run['display_name']
    samples_df['Throughput Rolling'] = samples_df[
        'Throughput (requests/second)'].rolling(5, min_periods=1).mean()

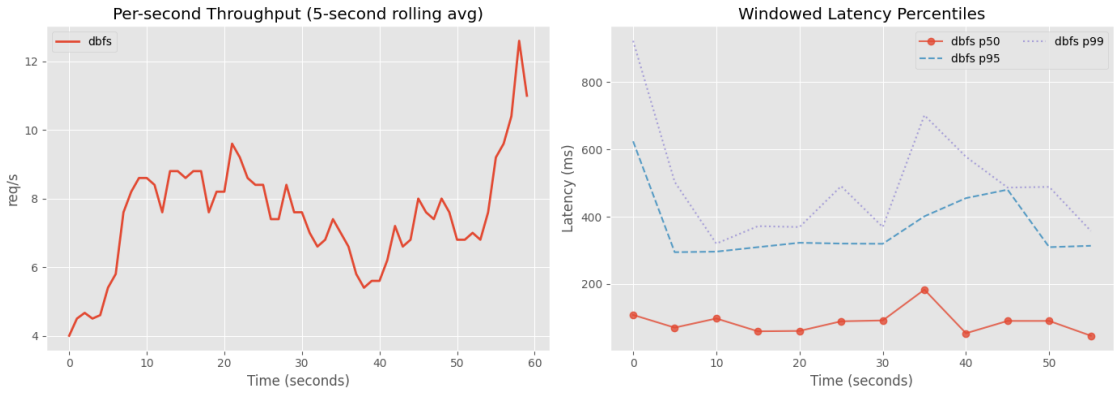
    axes[0].plot(
```

```
        samples_df['Time (seconds)'],
        samples_df['Throughput Rolling'],
        linewidth=2,
        label=label,
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['Median Latency (millisecond)'],
        marker='o',
        alpha=0.8,
        label=f'{label} p50',
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['95th Percentile Latency (millisecond)'],
        linestyle='--',
        alpha=0.8,
        label=f'{label} p95',
    )
    axes[1].plot(
        results_df['Time (seconds)'],
        results_df['99th Percentile Latency (millisecond)'],
        linestyle=':',
        alpha=0.8,
        label=f'{label} p99',
    )

axes[0].set_title('Per-second Throughput (5-second rolling avg)')
axes[0].set_xlabel('Time (seconds)')
axes[0].set_ylabel('req/s')
axes[0].legend()

axes[1].set_title('Windowed Latency Percentiles')
axes[1].set_xlabel('Time (seconds)')
axes[1].set_ylabel('Latency (ms)')
axes[1].legend(ncol=2)

fig.tight_layout()
plt.show()
```



[9]:
```
procedure_rows = []
for engine, run in runs.items():
    for procedure_name, procedure_df in run['procedure_dfs'].items():
        procedure_rows.append({
            'Engine':
            run['display_name'],
            'Procedure':
            procedure_name,
            'Avg Throughput (req/s)':
            procedure_df['Throughput (requests/second)'].mean(),
            'Avg Latency (ms)':
            procedure_df['Average Latency (millisecond)'].mean(),
            'Avg P95 Latency (ms)':
            procedure_df['95th Percentile Latency (millisecond)'].mean(),
        })

procedure_comparison_df = pd.DataFrame(procedure_rows).sort_values(
    ['Procedure', 'Engine'])
display(procedure_comparison_df)

procedure_throughput = procedure_comparison_df.pivot(
    index='Procedure', columns='Engine', values='Avg Throughput (req/s)')
procedure_p95 = procedure_comparison_df.pivot(index='Procedure',
                                              columns='Engine',
                                              values='Avg P95 Latency (ms)')

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
procedure_throughput.plot(kind='bar', ax=axes[0])
axes[0].set_title('Average Throughput by Procedure')
axes[0].set_ylabel('req/s')
axes[0].tick_params(axis='x', rotation=30)

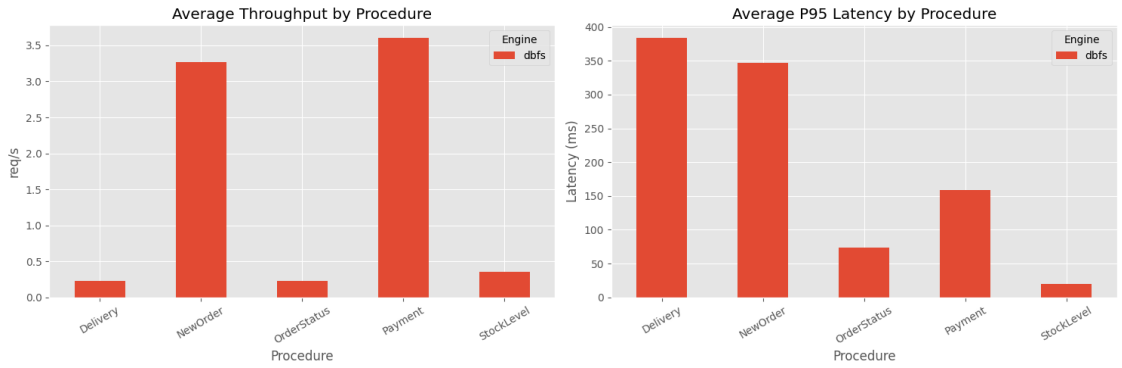
procedure_p95.plot(kind='bar', ax=axes[1])
```

```
axes[1].set_title('Average P95 Latency by Procedure')
axes[1].set_ylabel('Latency (ms)')
axes[1].tick_params(axis='x', rotation=30)

fig.tight_layout()
plt.show()
```

```
  Engine    Procedure  Avg Throughput (req/s)  Avg Latency (ms)  \
0   dbfs     Delivery                   0.233           364.305
1   dbfs     NewOrder                   3.267           215.621
2   dbfs  OrderStatus                   0.233            61.026
3   dbfs      Payment                   3.600            53.339
4   dbfs   StockLevel                   0.350            13.196

   Avg P95 Latency (ms)
0               383.508
1               346.591
2                73.883
3               158.300
4                20.343
```



[10]:
```
mix_comparison_rows = []
for engine in transaction_mix_df['Engine'].unique():
    engine_mix = transaction_mix_df[transaction_mix_df['Engine'] ==
                                    engine].set_index('Transaction')
    for transaction_name, configured_ratio in configured_ratios.items():
        observed_ratio = engine_mix.loc[transaction_name, 'Observed Mix (%)']
        mix_comparison_rows.append({
            'Engine':
            engine,
            'Transaction':
            transaction_name,
            'Configured Ratio (%)':
            configured_ratio,
```

```
            'Observed Ratio (%)':
            observed_ratio,
            'Absolute Error (pp)':
            abs(observed_ratio - configured_ratio),
        })

mix_comparison_df = pd.DataFrame(mix_comparison_rows)
display(mix_comparison_df)

report_rows = []
for engine, run in runs.items():
    parsed = parse_terminal_log(run['terminal_log'])
    completed_total = transaction_mix_df.loc[transaction_mix_df['Engine'] ==
                                             run['display_name'],
                                             'Completed'].sum()
    aborted_total = transaction_mix_df.loc[
        transaction_mix_df['Engine'] == run['display_name'], 'Aborted'].sum()

    report_rows.append({
        'run_label':
        run['label'],
        'engine':
        run['display_name'],
        'dbms_type':
        parsed['dbms_type'],
        'workload':
        parsed['workload'],
        'scale_factor':
        float(parsed['scale_factor']),
        'terminals':
        int(parsed['terminals']),
        'duration_seconds':
        run['summary']['Elapsed Time (nanoseconds)'] / 1_000_000_000,
        'measured_requests':
        run['summary']['Measured Requests'],
        'throughput_req_per_s':
        run['summary']['Throughput (requests/second)'],
        'goodput_req_per_s':
        run['summary']['Goodput (requests/second)'],
        'completed_transactions':
        int(completed_total),
        'aborted_transactions':
        int(aborted_total),
        'abort_percentage':
        (aborted_total / (completed_total + aborted_total) * 100) if
        (completed_total + aborted_total) else 0.0,
    })
```

```
report_df = pd.DataFrame(report_rows)
display(report_df)
```

```
  Engine  Transaction  Configured Ratio (%)  Observed Ratio (%)  \
0   dbfs     NewOrder                45.000               0.000
1   dbfs      Payment                43.000               0.000
2   dbfs  OrderStatus                 4.000               0.000
3   dbfs     Delivery                 4.000               0.000
4   dbfs   StockLevel                 4.000               0.000

   Absolute Error (pp)
0               45.000
1               43.000
2                4.000
3                4.000
4                4.000

                 run_label engine dbms_type workload  scale_factor  \
0  tpcc_2026-06-18_22-30-47   dbfs  POSTGRES     TPCC         1.000

   terminals  duration_seconds  measured_requests  throughput_req_per_s  \
0          1            60.000                461                 7.683

   goodput_req_per_s  completed_transactions  aborted_transactions  \
0              7.650                       0                     0

   abort_percentage
0             0.000
```

[11]:
```
def summarize_dbfs_query_trace(trace_df):
    summary_df = trace_df.copy()
    summary_df['sample_sql'] = summary_df['sample_sql'].fillna(
        summary_df['shape_key'])
    summary_df['total_elapsed_ms'] = summary_df['total_elapsed_us'] / 1000.0
    summary_df['mean_elapsed_ms'] = summary_df['mean_elapsed_us'] / 1000.0
    summary_df['p95_elapsed_ms'] = pd.NA
    summary_df['max_elapsed_ms'] = summary_df['max_elapsed_us'] / 1000.0
    summary_df = summary_df.sort_values(['total_elapsed_ms', 'executions'],
                                        ascending=[False, False])
    return summary_df[[
        'kind',
        'shape_key',
        'sample_sql',
        'executions',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
```

```
        'max_elapsed_ms',
        'failures',
    ]]


def summarize_postgres_server_query_trace(trace_df):
    summary_df = trace_df.copy()
    summary_df['kind'] = 'QUERY'
    summary_df['shape_key'] = summary_df['query'].str.replace(
        r'\s+', ' ', regex=True).str.strip()
    summary_df['sample_sql'] = summary_df['query']
    summary_df['executions'] = summary_df['calls']
    summary_df['total_elapsed_ms'] = summary_df['total_exec_time']
    summary_df['mean_elapsed_ms'] = summary_df['mean_exec_time']
    summary_df['p95_elapsed_ms'] = pd.NA
    summary_df['max_elapsed_ms'] = summary_df['max_exec_time']
    summary_df['failures'] = 0
    summary_df = summary_df.sort_values(['total_elapsed_ms', 'executions'],
                                        ascending=[False, False])
    return summary_df[[
        'kind',
        'shape_key',
        'sample_sql',
        'executions',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
    ]]


def summarize_postgres_query_trace(trace_df):
    if {
            'query', 'calls', 'total_exec_time', 'mean_exec_time',
            'max_exec_time'
    }.issubset(trace_df.columns):
        return summarize_postgres_server_query_trace(trace_df)
    return summarize_dbfs_query_trace(trace_df)


def escape_plot_label(text):
    return text.replace('$', r'\$')


def build_shared_trace_comparison(trace_summaries):
    if not {'dbfs', 'postgres'}.issubset(trace_summaries):
```

```
        return None

    shared_df = trace_summaries['dbfs'].merge(
        trace_summaries['postgres'],
        on='shape_key',
        how='inner',
        suffixes=('_dbfs', '_postgres'),
    )
    if shared_df.empty:
        return shared_df

    shared_df['sample_sql'] = shared_df['sample_sql_dbfs'].fillna(
        shared_df['sample_sql_postgres'])
    shared_df['combined_total_elapsed_ms'] = (
        shared_df['total_elapsed_ms_dbfs'] +
        shared_df['total_elapsed_ms_postgres'])
    shared_df['combined_max_elapsed_ms'] = shared_df[[
        'max_elapsed_ms_dbfs',
        'max_elapsed_ms_postgres',
    ]].max(axis=1)
    shared_df['label'] = shared_df['sample_sql'].str.slice(0, 96)
    shared_df['plot_label'] = shared_df['label'].map(escape_plot_label)
    return shared_df.sort_values(
        ['combined_total_elapsed_ms', 'combined_max_elapsed_ms'],
        ascending=[False, False],
    )


trace_summaries = {}
for engine, run in runs.items():
    trace_df = run['query_trace_df']
    if trace_df is None or trace_df.empty:
        print(
            f"{run['display_name']}: {query_trace_filenames[engine]} was not␣
 ↪found."
        )
        continue

    if engine == 'dbfs':
        engine_summary_df = summarize_dbfs_query_trace(trace_df.copy())
    else:
        engine_summary_df = summarize_postgres_query_trace(trace_df.copy())

    engine_summary_df['Engine'] = run['display_name']
    engine_summary_df['time_share_pct'] = (
        engine_summary_df['total_elapsed_ms'] /
        engine_summary_df['total_elapsed_ms'].sum() * 100)
```

```
    engine_summary_df['label'] = engine_summary_df['sample_sql'].str.slice(
        0, 96)
    engine_summary_df['plot_label'] = engine_summary_df['label'].map(
        escape_plot_label)
    trace_summaries[engine] = engine_summary_df

for engine, summary_df in trace_summaries.items():
    display_name = runs[engine]['display_name']
    print(f'Top query shapes for {display_name}')
    display(summary_df[[
        'kind',
        'executions',
        'time_share_pct',
        'total_elapsed_ms',
        'mean_elapsed_ms',
        'p95_elapsed_ms',
        'max_elapsed_ms',
        'failures',
        'sample_sql',
    ]].head(15))

shared_trace_df = build_shared_trace_comparison(trace_summaries)
if shared_trace_df is not None:
    overlap_summary_df = pd.DataFrame([{
        'dbfs_shapes':
        len(trace_summaries['dbfs']),
        'postgres_shapes':
        len(trace_summaries['postgres']),
        'shared_shapes':
        len(shared_trace_df),
        'dbfs_only':
        len(trace_summaries['dbfs']) - len(shared_trace_df),
        'postgres_only':
        len(trace_summaries['postgres']) - len(shared_trace_df),
    }])
    print('Shared query shape overlap')
    display(overlap_summary_df)

    display(shared_trace_df[[
        'kind_dbfs',
        'executions_dbfs',
        'executions_postgres',
        'total_elapsed_ms_dbfs',
        'total_elapsed_ms_postgres',
        'max_elapsed_ms_dbfs',
        'max_elapsed_ms_postgres',
        'sample_sql',
```

```
    ]].head(15))

    fig, axes = plt.subplots(1, 2, figsize=(16, 8))

    plot_specs = [
        (
            'combined_total_elapsed_ms',
            'total_elapsed_ms',
            'Shared Query Shapes by Total Time',
        ),
        (
            'combined_max_elapsed_ms',
            'max_elapsed_ms',
            'Shared Query Shapes by Max Latency',
        ),
    ]

    for axis, (combined_metric, metric, title) in zip(axes, plot_specs):
        top_df = shared_trace_df.nlargest(
            10, combined_metric).sort_values(combined_metric)
        positions = list(range(len(top_df)))
        axis.barh(
            [position - 0.2 for position in positions],
            top_df[f'{metric}_dbfs'],
            height=0.4,
            label='dbfs',
            alpha=0.8,
        )
        axis.barh(
            [position + 0.2 for position in positions],
            top_df[f'{metric}_postgres'],
            height=0.4,
            label='PostgreSQL',
            alpha=0.8,
        )
        axis.set_yticks(positions)
        axis.set_yticklabels(top_df['plot_label'])
        axis.set_title(title)
        axis.set_xlabel('Latency (ms)')

    axes[0].legend()
    fig.tight_layout()
    plt.show()
```

```
Top query shapes for dbfs

            kind  executions  time_share_pct  total_elapsed_ms  \
0          QUERY        2091          20.982        12,225.844
```

1	UPDATE	2057	20.193	11,766.430
2	QUERY	2088	17.841	10,396.026
3	UPDATE	195	5.827	3,395.480
4	UPDATE	140	3.033	1,767.527
5	QUERY	140	2.995	1,745.064
6	UPDATE	216	2.626	1,530.418
7	QUERY	140	2.351	1,369.694
8	BATCH_UPDATE	194	2.350	1,369.161
9	QUERY	197	1.764	1,027.769
10	QUERY	197	1.728	1,007.107
11	UPDATE	216	1.549	902.752
12	UPDATE	216	1.518	884.746
13	QUERY	197	1.359	791.908
14	UPDATE	140	1.329	774.579

	mean_elapsed_ms	p95_elapsed_ms	max_elapsed_ms	failures	\
0	5.846	<NA>	78.812	0	
1	5.720	<NA>	48.002	0	
2	4.978	<NA>	47.460	0	
3	17.412	<NA>	45.710	0	
4	12.625	<NA>	44.526	0	
5	12.464	<NA>	44.523	0	
6	7.085	<NA>	531.449	0	
7	9.783	<NA>	157.798	0	
8	7.057	<NA>	51.975	0	
9	5.217	<NA>	46.107	0	
10	5.112	<NA>	45.428	0	
11	4.179	<NA>	45.860	0	
12	4.096	<NA>	44.447	0	
13	4.019	<NA>	42.695	0	
14	5.532	<NA>	44.198	0	

	sample_sql
0	SELECT I_PRICE, I_NAME , I_DATA FROM item WHER…
1	UPDATE stock SET S_QUANTITY = 27 , S_YTD = S_Y…
2	SELECT S_QUANTITY, S_DATA, S_DIST_01, S_DIST_0…
3	UPDATE customer SET C_BALANCE = -3806.12011718…
4	DELETE FROM new_order WHERE NO_O_ID = 2101 AND…
5	SELECT SUM(OL_AMOUNT) AS OL_TOTAL FROM order_l…
6	UPDATE warehouse SET W_YTD = W_YTD + 4008.6599…
7	SELECT C_FIRST, C_MIDDLE, C_ID, C_STREET_1, C_…
8	INSERT INTO order_line (OL_O_ID, OL_D_ID, OL_W…
9	SELECT C_DISCOUNT, C_LAST, C_CREDIT FROM custo…
10	SELECT W_TAX FROM warehouse WHERE W_ID = 1
11	UPDATE district SET D_YTD = D_YTD + 4008.65991…
12	INSERT INTO history (H_C_D_ID, H_C_W_ID, H_C_I…
13	SELECT D_NEXT_O_ID, D_TAX FROM district WHERE …
14	UPDATE order_line SET OL_DELIVERY_D = '2026-06…

1.1 dbfs Buffer Pool Stats

This section reads `buffer_pool_stats` lines from the dbfs benchmark logs under `stats_comparison_root`. The C++ side emits raw cumulative counters at a time-based interval. The notebook slices rows to the BenchBase execute interval, subtracts the last pre-execute row as the baseline, and visualizes per-log-interval `delta_*` values on an elapsed-seconds x-axis.

- `pin_page_calls`: cumulative calls to `BufferPool::pinPage`; this is the broad request volume seen by the buffer pool.
- `resident_hits`: cumulative page requests served from an already resident frame.
- `misses`: cumulative page requests that required loading or allocating a frame.
- `evictions`: cumulative pages evicted from the buffer pool.
- `dirty_evictions`: cumulative evictions that required writing a dirty page.
- `reads`: cumulative `readPageIntoBuffer` calls.
- `zero_outs`: cumulative frame zeroing calls.
- `hit_rate_pct`: `resident_hits / (resident_hits + misses) * 100`.
- `miss_rate_pct`: `misses / (resident_hits + misses) * 100`; this is the complement of `hit_rate_pct`, so the graph shows hit rate only.
- `evictions_per_miss`: `evictions / misses`; after the buffer pool is full, this approaches 1 when each miss needs to evict a frame.
- `delta_evictions_per_delta_miss`: `delta_evictions / delta_misses`; this shows the same relationship per emitted log interval.
- `dirty_eviction_rate_pct`: `dirty_evictions / evictions * 100`.
- `delta_dirty_eviction_rate_pct`: `delta_dirty_evictions / delta_evictions * 100`; this shows how often evictions in each interval required flushing a dirty page.
- `delta_*`: increment since the previous emitted log row.

[12]:
```
buffer_pool_stats_pattern = re.compile(r'buffer_pool_stats\s+(?P<stats>.*)$')
key_value_pattern = re.compile(r'(\w+)=(\d+)')
buffer_pool_counter_columns = [
    'pin_page_calls',
    'resident_hits',
    'misses',
    'evictions',
    'dirty_evictions',
    'reads',
    'zero_outs',
]
buffer_pool_snapshot_columns = [
    'frames_free',
    'frames_resident',
    'frames_pinned',
    'frames_evictable',
    'resident_heap_pages',
    'resident_leaf_index_pages',
    'resident_internal_index_pages',
    'pinned_heap_pages',
    'pinned_leaf_index_pages',
```

```
    'pinned_internal_index_pages',
    'dirty_heap_pages',
    'dirty_leaf_index_pages',
    'dirty_internal_index_pages',
]
buffer_pool_stat_columns = (buffer_pool_counter_columns +
                            buffer_pool_snapshot_columns)
compose_timestamp_pattern = re.compile(
    r'(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d+)?Z)')
benchbase_timestamp_pattern = re.compile(
    r'\[INFO \]\s+(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2},\d{3})')


def find_buffer_pool_log_paths(result_dir):
    candidates = [
        result_dir / 'dbfs.compose.log',
        result_dir / 'benchbase_execute.log',
        result_dir / 'benchbase.log',
    ]
    candidates.extend(sorted(result_dir.glob('*.log')))

    seen = set()
    paths = []
    for path in candidates:
        if path in seen or not path.exists():
            continue
        seen.add(path)
        paths.append(path)
    return paths


def parse_compose_timestamp(line):
    match = compose_timestamp_pattern.search(line)
    if not match:
        return pd.NaT
    return pd.to_datetime(match.group(1), utc=True)


def parse_benchbase_timestamp(line):
    match = benchbase_timestamp_pattern.search(line)
    if not match:
        return pd.NaT
    return pd.to_datetime(match.group(1).replace(',', '.'), utc=True)


def find_benchbase_execute_interval(result_dir):
    log_paths = [
```

```
        result_dir / 'benchbase_execute.log', result_dir / 'benchbase.log'
    ]
    execute_log_path = next((path for path in log_paths if path.exists()),
                            None)
    if execute_log_path is None:
        return None, None

    start_at = None
    end_at = None
    with execute_log_path.open(encoding='utf-8', errors='replace') as handle:
        for line in handle:
            timestamp = parse_benchbase_timestamp(line)
            if pd.isna(timestamp):
                continue
            if 'MEASURE' in line and start_at is None:
                start_at = timestamp
            if 'TERMINATE' in line:
                end_at = timestamp
                break
    return start_at, end_at


def add_buffer_pool_derived_columns(stats_df):
    numeric_columns = [
        column for column in buffer_pool_stat_columns
        if column in stats_df.columns
    ]
    for column in numeric_columns:
        stats_df[column] = pd.to_numeric(stats_df[column])
    for column in buffer_pool_counter_columns:
        if column in stats_df.columns:
            stats_df[f'delta_{column}'] = stats_df[column].diff().fillna(
                stats_df[column])
    stats_df['sample_index'] = stats_df.index + 1
    if 'logged_at' in stats_df.columns:
        first_logged_at = stats_df['logged_at'].dropna().min()
        if pd.notna(first_logged_at):
            stats_df['elapsed_seconds'] = (stats_df['logged_at'] -
                                           first_logged_at).dt.total_seconds()

    requests = stats_df['resident_hits'] + stats_df['misses']
    stats_df['hit_rate_pct'] = (stats_df['resident_hits'] / requests *
                                100).where(requests > 0)
    stats_df['miss_rate_pct'] = (stats_df['misses'] / requests *
                                 100).where(requests > 0)
    stats_df['evictions_per_miss'] = (stats_df['evictions'] /
                                      stats_df['misses']).where(
```

```
                                          stats_df['misses'] > 0)
    stats_df['delta_evictions_per_delta_miss'] = (
        stats_df['delta_evictions'] /
        stats_df['delta_misses']).where(stats_df['delta_misses'] > 0)
    stats_df['dirty_eviction_rate_pct'] = (stats_df['dirty_evictions'] /
                                           stats_df['evictions'] * 100).where(
                                               stats_df['evictions'] > 0)

    delta_requests = stats_df['delta_resident_hits'] + stats_df['delta_misses']
    stats_df['delta_hit_rate_pct'] = (stats_df['delta_resident_hits'] /
                                      delta_requests *
                                      100).where(delta_requests > 0)
    stats_df['delta_miss_rate_pct'] = (stats_df['delta_misses'] /
                                       delta_requests *
                                       100).where(delta_requests > 0)
    stats_df['delta_dirty_eviction_rate_pct'] = (
        stats_df['delta_dirty_evictions'] / stats_df['delta_evictions'] *
        100).where(stats_df['delta_evictions'] > 0)

    if 'frames_resident' in stats_df.columns:
        stats_df['pinned_frame_rate_pct'] = (
            stats_df['frames_pinned'] / stats_df['frames_resident'] *
            100).where(stats_df['frames_resident'] > 0)

    dirty_page_columns = [
        'dirty_heap_pages',
        'dirty_leaf_index_pages',
        'dirty_internal_index_pages',
    ]
    if set(dirty_page_columns).issubset(stats_df.columns):
        stats_df['dirty_frames'] = stats_df[dirty_page_columns].sum(axis=1)
    return stats_df


def subtract_buffer_pool_baseline(stats_df, baseline_row):
    adjusted_df = stats_df.copy()
    if baseline_row is None:
        return adjusted_df

    for column in buffer_pool_counter_columns:
        if column in adjusted_df.columns and column in baseline_row:
            adjusted_df[column] = adjusted_df[column] - baseline_row[column]
    return adjusted_df


def load_buffer_pool_stats(result_dir):
    rows = []
```

```
    for log_path in find_buffer_pool_log_paths(result_dir):
        with log_path.open(encoding='utf-8', errors='replace') as handle:
            for line_number, line in enumerate(handle, start=1):
                match = buffer_pool_stats_pattern.search(line)
                if not match:
                    continue

                parsed_stats = {
                    key: int(value)
                    for key, value in key_value_pattern.findall(
                        match.group('stats'))
                }
                row = {
                    key: parsed_stats[key]
                    for key in buffer_pool_stat_columns if key in parsed_stats
                }
                row['log_file'] = log_path.name
                row['line_number'] = line_number
                row['logged_at'] = parse_compose_timestamp(line)
                rows.append(row)

    if not rows:
        return pd.DataFrame()

    return pd.DataFrame(rows).sort_values(
        ['logged_at', 'evictions', 'line_number']).reset_index(drop=True)


def select_execute_buffer_pool_stats(stats_df, result_dir):
    start_at, end_at = find_benchbase_execute_interval(result_dir)
    if start_at is None or end_at is None or 'logged_at' not in stats_df:
        return add_buffer_pool_derived_columns(
            stats_df.copy()), start_at, end_at

    baseline_rows = stats_df[stats_df['logged_at'] < start_at]
    baseline_row = None if baseline_rows.empty else baseline_rows.iloc[-1]
    execute_df = stats_df[(stats_df['logged_at'] >= start_at)
                          & (stats_df['logged_at'] <= end_at)].copy()
    execute_df = subtract_buffer_pool_baseline(execute_df, baseline_row)
    execute_df['execute_elapsed_seconds'] = (execute_df['logged_at'] -
                                             start_at).dt.total_seconds()
    execute_df = execute_df.reset_index(drop=True)
    return add_buffer_pool_derived_columns(execute_df), start_at, end_at


print(f'Loading buffer pool statistics from: {stats_dbfs_result_dir}')
raw_buffer_pool_stats_df = load_buffer_pool_stats(stats_dbfs_result_dir)
```

```
if raw_buffer_pool_stats_df.empty:
    buffer_pool_stats_df = raw_buffer_pool_stats_df
    execute_start_at = None
    execute_end_at = None
else:
    buffer_pool_stats_df, execute_start_at, execute_end_at = (
        select_execute_buffer_pool_stats(raw_buffer_pool_stats_df,
                                         stats_dbfs_result_dir))

if buffer_pool_stats_df.empty:
    print('No buffer_pool_stats rows found. Run dbfs with '
          'DBFS_BUFFER_POOL_LOG_STATS_EVERY_MS > 0 and expanded result logs.')
else:
    if execute_start_at is not None and execute_end_at is not None:
        print(f'Using BenchBase execute interval: {execute_start_at} to '
              f'{execute_end_at}.')
        print('Displayed counters are relative to the last stats row before '
              'the execute interval.')
    else:
        print('BenchBase execute interval was not found; displaying all '
              'available buffer_pool_stats rows.')
    print('Graphs use delta_* columns, so each point shows the increase '
          'since the previous displayed log row.')
    elapsed_x_column = 'execute_elapsed_seconds'
    if elapsed_x_column not in buffer_pool_stats_df.columns:
        elapsed_x_column = 'elapsed_seconds'
    print(f'The x-axis is {elapsed_x_column}.')
    print('evictions_per_miss and delta_evictions_per_delta_miss show how '
          'often page misses turn into evictions.')
    display(buffer_pool_stats_df[[
        'sample_index',
        'logged_at',
        elapsed_x_column,
        'delta_pin_page_calls',
        'delta_resident_hits',
        'delta_misses',
        'delta_evictions',
        'delta_reads',
        'delta_dirty_evictions',
        'delta_hit_rate_pct',
        'delta_evictions_per_delta_miss',
        'delta_dirty_eviction_rate_pct',
        'frames_resident',
        'frames_free',
        'frames_pinned',
        'frames_evictable',
        'dirty_frames',
```

```
    'pinned_frame_rate_pct',
]].tail(10))

plot_buffer_pool_stats_df = buffer_pool_stats_df[
    buffer_pool_stats_df[elapsed_x_column] >= 3.0].copy()
if plot_buffer_pool_stats_df.empty:
    plot_buffer_pool_stats_df = buffer_pool_stats_df.copy()
else:
    print('Plots exclude the first 3 seconds because startup samples '
          'can include setup-to-execute boundary effects.')

# Rename columns to remove 'delta_' prefix for cleaner labels
plot_df = plot_buffer_pool_stats_df.copy()
plot_df = plot_df.rename(
    columns={
        'delta_pin_page_calls': 'Total Page Requests',
        'delta_misses': 'Misses',
        'delta_miss_rate_pct': 'Miss Rate (%)',
        'delta_dirty_eviction_rate_pct': 'Dirty Eviction Rate (%)'
    })

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Left: request/miss counts and miss rate
ax_left = axes[0]
ax_left_twin = ax_left.twinx()

x_values = plot_df[elapsed_x_column]
ax_left.bar(x_values,
            plot_df['Total Page Requests'],
            width=0.8,
            color='lightgray',
            label='Total Page Requests')
ax_left.bar(x_values,
            plot_df['Misses'],
            width=0.5,
            color='tab:orange',
            label='Misses')
ax_left_twin.plot(x_values,
                  plot_df['Miss Rate (%)'],
                  color='tab:blue',
                  marker='o',
                  markersize=3,
                  label='Miss Rate (%)')
ax_left.set_title('Buffer Pool: Requests and Misses')
ax_left.set_xlabel('Elapsed seconds')
ax_left.set_ylabel('Calls')
```

```
ax_left_twin.set_ylabel('Miss Rate (%)')

# Combine legends from both axes and position outside plot
lines1, labels1 = ax_left.get_legend_handles_labels()
lines2, labels2 = ax_left_twin.get_legend_handles_labels()
ax_left.legend(lines1 + lines2,
               labels1 + labels2,
               loc='upper left',
               bbox_to_anchor=(0, 1),
               frameon=True)

# Right: Dirty eviction rate
plot_df.plot(
    x=elapsed_x_column,
    y=['Dirty Eviction Rate (%)'],
    ax=axes[1],
    color='tab:green',
    marker='o',
    markersize=3,
    legend=False,
)
axes[1].set_title('Dirty Eviction Rate per Log Interval')
axes[1].set_xlabel('Elapsed seconds')
axes[1].set_ylabel('% of evictions')
axes[1].legend(['Dirty Eviction Rate (%)'], loc='best')

fig.tight_layout()
plt.show()

frame_state_columns = [
    'frames_evictable',
    'frames_free',
    'frames_pinned',
    'dirty_frames',
]
page_kind_columns = [
    'resident_heap_pages',
    'resident_leaf_index_pages',
    'resident_internal_index_pages',
]
dirty_page_kind_columns = [
    'dirty_heap_pages',
    'dirty_leaf_index_pages',
    'dirty_internal_index_pages',
]
frame_label_map = {
    'frames_evictable': 'evictable',
```

```
        'frames_free': 'free',
        'frames_pinned': 'pinned',
        'dirty_frames': 'dirty',
        'pinned_frame_rate_pct': 'pinned rate (%)',
    }
    page_kind_label_map = {
        'resident_heap_pages': 'heap',
        'resident_leaf_index_pages': 'leaf index',
        'resident_internal_index_pages': 'internal index',
        'dirty_heap_pages': 'dirty heap',
        'dirty_leaf_index_pages': 'dirty leaf index',
        'dirty_internal_index_pages': 'dirty internal index',
    }

    required_frame_columns = set(frame_state_columns + page_kind_columns +
                                 dirty_page_kind_columns)
    if required_frame_columns.issubset(plot_df.columns):
        fig, axes = plt.subplots(1, 2, figsize=(16, 5))

        frame_style_map = {
            'evictable': {
                'color': 'tab:blue',
                'linestyle': '-'
            },
            'free': {
                'color': 'tab:gray',
                'linestyle': ':'
            },
            'pinned': {
                'color': 'tab:red',
                'linestyle': '-.',
                'marker': 'o',
                'markersize': 3,
            },
            'dirty': {
                'color': 'tab:orange',
                'linestyle': '--'
            },
        }
        frame_state_plot_df = plot_df[[elapsed_x_column] +
                                      frame_state_columns].rename(
                                          columns=frame_label_map)
        for column in frame_state_plot_df.columns:
            if column == elapsed_x_column:
                continue
            style = frame_style_map.get(column, {})
            axes[0].plot(frame_state_plot_df[elapsed_x_column],
```

```
                    frame_state_plot_df[column],
                    label=column,
                    linewidth=1.8,
                    **style)
    axes[0].set_title('Buffer Pool Frames by State')
    axes[0].set_xlabel('Elapsed seconds')
    axes[0].set_ylabel('Frames')
    axes[0].grid(True, alpha=0.25)

    if 'pinned_frame_rate_pct' in plot_df.columns:
        pinned_axis = axes[0].twinx()
        pinned_axis.plot(plot_df[elapsed_x_column],
                         plot_df['pinned_frame_rate_pct'],
                         label='pinned rate (%)',
                         color='black',
                         linestyle='--',
                         linewidth=1.4,
                         alpha=0.7)
        pinned_axis.set_ylabel('Pinned rate (%)')
        max_rate = plot_df['pinned_frame_rate_pct'].max(skipna=True)
        if pd.notna(max_rate):
            pinned_axis.set_ylim(0, max(5, min(100, max_rate * 1.2)))
        lines1, labels1 = axes[0].get_legend_handles_labels()
        lines2, labels2 = pinned_axis.get_legend_handles_labels()
        axes[0].legend(lines1 + lines2,
                       labels1 + labels2,
                       loc='best',
                       frameon=True)
    else:
        axes[0].legend(loc='best', frameon=True)

    resident_style_map = {
        'heap': 'tab:blue',
        'leaf index': 'tab:green',
        'internal index': 'tab:purple',
    }
    dirty_style_map = {
        'dirty heap': 'tab:blue',
        'dirty leaf index': 'tab:green',
        'dirty internal index': 'tab:purple',
    }
    page_kind_plot_df = plot_df[[elapsed_x_column] +
                                page_kind_columns].rename(
                                    columns=page_kind_label_map)
    dirty_page_kind_plot_df = plot_df[[elapsed_x_column] +
                                      dirty_page_kind_columns].rename(
                                          columns=page_kind_label_map)
```

```
        for column in page_kind_plot_df.columns:
            if column == elapsed_x_column:
                continue
            axes[1].plot(page_kind_plot_df[elapsed_x_column],
                         page_kind_plot_df[column],
                         label=column,
                         color=resident_style_map.get(column),
                         linestyle='-',
                         linewidth=1.8)
        for column in dirty_page_kind_plot_df.columns:
            if column == elapsed_x_column:
                continue
            axes[1].plot(dirty_page_kind_plot_df[elapsed_x_column],
                         dirty_page_kind_plot_df[column],
                         label=column,
                         color=dirty_style_map.get(column),
                         linestyle='--',
                         linewidth=1.6)
        axes[1].set_title('Resident and Dirty Pages by Page Kind')
        axes[1].set_xlabel('Elapsed seconds')
        axes[1].set_ylabel('Pages')
        axes[1].grid(True, alpha=0.25)
        axes[1].legend(loc='best', frameon=True)

        fig.tight_layout()
        plt.show()
    else:
        missing = sorted(required_frame_columns - set(plot_df.columns))
        print('Skipping frame/page-kind plots; missing columns: ' +
              ', '.join(missing))
```

```
Loading buffer pool statistics from:
/Users/yohasebe/Documents/dbfs/benchmarking/results/20260619-074745/tpcc/dbfs
Using BenchBase execute interval: 2026-06-18 22:53:13.606000+00:00 to 2026-06-18
22:54:13.613000+00:00.
Displayed counters are relative to the last stats row before the execute
interval.
Graphs use delta_* columns, so each point shows the increase since the previous
displayed log row.
The x-axis is execute_elapsed_seconds.
evictions_per_miss and delta_evictions_per_delta_miss show how often page misses
turn into evictions.

    sample_index                      logged_at  execute_elapsed_seconds  \
49            50 2026-06-18 22:54:04.084233834+00:00                   50.478
50            51 2026-06-18 22:54:05.104448251+00:00                   51.498
51            52 2026-06-18 22:54:06.142184377+00:00                   52.536
52            53 2026-06-18 22:54:07.148919419+00:00                   53.543
```

```
53           54 2026-06-18 22:54:08.170518211+00:00                  54.565
54           55 2026-06-18 22:54:09.172234045+00:00                  55.566
55           56 2026-06-18 22:54:10.196389712+00:00                  56.590
56           57 2026-06-18 22:54:11.204065837+00:00                  57.598
57           58 2026-06-18 22:54:12.199915130+00:00                  58.594
58           59 2026-06-18 22:54:13.200417505+00:00                  59.594

    delta_pin_page_calls  delta_resident_hits  delta_misses  delta_evictions  \
49             5,243.000            5,217.000        26.000           30.000
50             4,559.000            4,505.000        54.000           58.000
51             4,550.000            4,526.000        24.000           28.000
52            14,639.000           14,609.000        30.000           32.000
53             4,565.000            4,537.000        28.000           33.000
54            12,385.000           12,375.000        10.000           16.000
55            12,779.000           12,774.000         5.000            8.000
56             5,480.000            5,459.000        21.000           25.000
57            16,811.000           16,810.000         1.000            7.000
58            15,815.000           15,651.000       164.000          169.000

    delta_reads  delta_dirty_evictions  delta_hit_rate_pct  \
49       26.000                 30.000              99.504
50       54.000                 58.000              98.816
51       24.000                 28.000              99.473
52       30.000                 32.000              99.795
53       28.000                 33.000              99.387
54       10.000                 16.000              99.919
55        5.000                  8.000              99.961
56       21.000                 25.000              99.617
57        1.000                  7.000              99.994
58      164.000                169.000              98.963

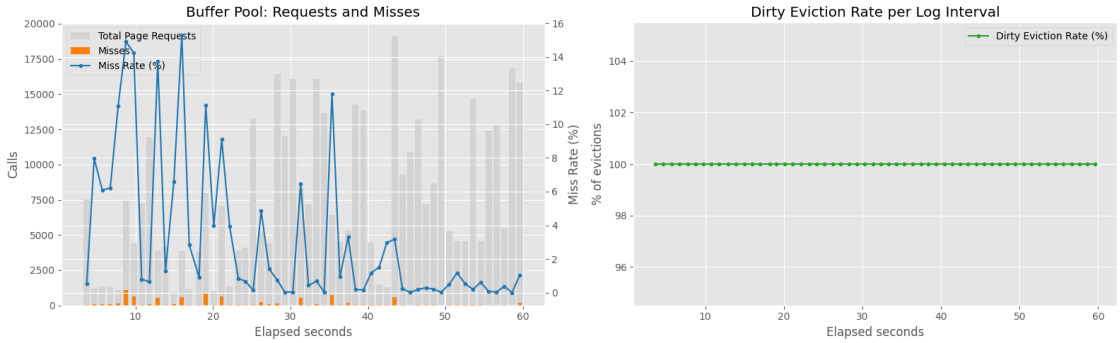
    delta_evictions_per_delta_miss  delta_dirty_eviction_rate_pct  \
49                           1.154                        100.000
50                           1.074                        100.000
51                           1.167                        100.000
52                           1.067                        100.000
53                           1.179                        100.000
54                           1.600                        100.000
55                           1.600                        100.000
56                           1.190                        100.000
57                           7.000                        100.000
58                           1.030                        100.000

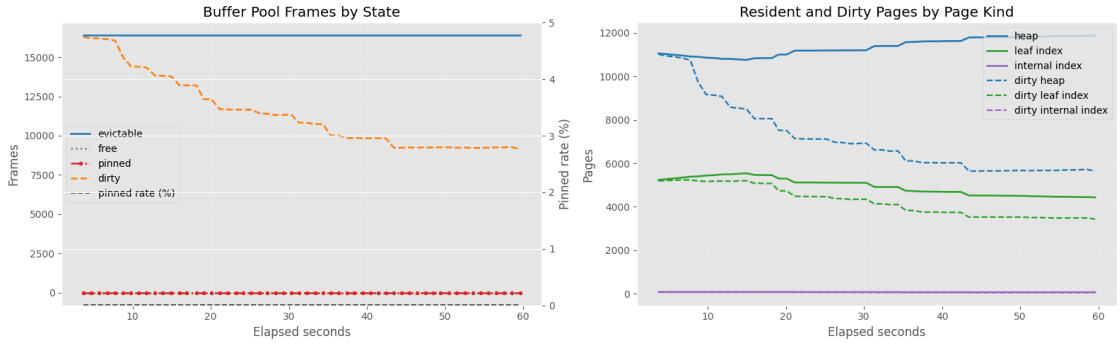
    frames_resident  frames_free  frames_pinned  frames_evictable  \
49            16384            0              1             16383
50            16384            0              1             16383
51            16384            0              1             16383
52            16384            0              1             16383
```

53	16384	0	1	16383
54	16384	0	1	16383
55	16384	0	1	16383
56	16384	0	1	16383
57	16384	0	1	16383
58	16384	0	1	16383

	dirty_frames	pinned_frame_rate_pct
49	9254	0.006
50	9232	0.006
51	9237	0.006
52	9227	0.006
53	9220	0.006
54	9240	0.006
55	9250	0.006
56	9257	0.006
57	9274	0.006
58	9130	0.006

Plots exclude the first 3 seconds because startup samples can include setup-to-execute boundary effects.





1.2 dbfs Buffer Pool Page Events

This section reads raw `buffer_pool_event` lines from `stats_comparison_root`. Each row is a page read or eviction event with page id, frame id, page kind, and dirty state. The standard `--stats` mode enables these high-volume events, while `--throughput` disables them.

[13]:
```
buffer_pool_event_unix_ms_pattern = re.compile(r'logged_at_unix_ms=(\d+)')
buffer_pool_event_pattern = re.compile(
    r'buffer_pool_event\s+type=(?P<type>\w+)\s+'
    r'event_id=(?P<event_id>\d+)\s+'
    r'file=(?P<file>\S+)\s+'
    r'page_id=(?P<page_id>\d+)\s+'
    r'frame_id=(?P<frame_id>\d+)\s+'
    r'page_kind=(?P<page_kind>\w+)\s+'
    r'dirty=(?P<dirty>[01])'
    r'(?:\s+hit=(?P<hit>[01]))?')

if 'find_buffer_pool_log_paths' not in globals():
    compose_timestamp_pattern = re.compile(
        r'(\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}(?:\.\d+)?Z)')
    benchbase_timestamp_pattern = re.compile(
        r'\[INFO \]\s+(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2},\d{3})')

    def find_buffer_pool_log_paths(result_dir):
        candidates = [
            result_dir / 'dbfs.compose.log',
            result_dir / 'benchbase_execute.log',
            result_dir / 'benchbase.log',
        ]
        candidates.extend(sorted(result_dir.glob('*.log')))
        seen = set()
        paths = []
        for path in candidates:
            if path in seen or not path.exists():
                continue
            seen.add(path)
            paths.append(path)
        return paths

    def parse_compose_timestamp(line):
        match = compose_timestamp_pattern.search(line)
        if not match:
            return pd.NaT
        return pd.to_datetime(match.group(1), utc=True)

    def parse_benchbase_timestamp(line):
        match = benchbase_timestamp_pattern.search(line)
        if not match:
```

```
            return pd.NaT
        return pd.to_datetime(match.group(1).replace(',', '.'), utc=True)

    def find_benchbase_execute_interval(result_dir):
        log_paths = [
            result_dir / 'benchbase_execute.log', result_dir / 'benchbase.log'
        ]
        execute_log_path = next((path for path in log_paths if path.exists()),
                                None)
        if execute_log_path is None:
            return None, None
        start_at = None
        end_at = None
        with execute_log_path.open(encoding='utf-8',
                                   errors='replace') as handle:
            for line in handle:
                timestamp = parse_benchbase_timestamp(line)
                if pd.isna(timestamp):
                    continue
                if 'MEASURE' in line and start_at is None:
                    start_at = timestamp
                if 'TERMINATE' in line:
                    end_at = timestamp
                    break
        return start_at, end_at


def find_buffer_pool_event_log_paths(result_dir):
    dedicated_log_path = result_dir / 'dbfs.buffer_pool_events.log'
    if dedicated_log_path.exists() and dedicated_log_path.stat().st_size > 0:
        return [dedicated_log_path]
    return find_buffer_pool_log_paths(result_dir)


def parse_buffer_pool_event_timestamp(line):
    unix_ms_match = buffer_pool_event_unix_ms_pattern.search(line)
    if unix_ms_match:
        return pd.to_datetime(int(unix_ms_match.group(1)), unit='ms', utc=True)
    timestamp = parse_compose_timestamp(line)
    if pd.isna(timestamp):
        timestamp = parse_benchbase_timestamp(line)
    return timestamp


def load_buffer_pool_events(result_dir):
    rows = []
    for log_path in find_buffer_pool_event_log_paths(result_dir):
```

```
        with log_path.open(encoding='utf-8', errors='replace') as handle:
            for line_number, line in enumerate(handle, start=1):
                match = buffer_pool_event_pattern.search(line)
                if not match:
                    continue
                timestamp = parse_buffer_pool_event_timestamp(line)
                row = match.groupdict()
                row.update({
                    'logged_at': timestamp,
                    'log_path': log_path.name,
                    'line_number': line_number,
                })
                rows.append(row)
    if not rows:
        return pd.DataFrame()
    events_df = pd.DataFrame(rows)
    for column in ['event_id', 'page_id', 'frame_id', 'dirty', 'hit']:
        if column in events_df.columns:
            events_df[column] = pd.to_numeric(events_df[column])
    events_df = events_df.sort_values(['event_id',
                                       'line_number']).reset_index(drop=True)
    events_df['page_key'] = (events_df['file'].astype(str) + ':' +
                             events_df['page_id'].astype(str))
    return events_df


def select_execute_buffer_pool_events(events_df, result_dir):
    start_at, end_at = find_benchbase_execute_interval(result_dir)
    if events_df.empty or start_at is None or end_at is None:
        selected_df = events_df.copy()
    else:
        selected_df = events_df[(events_df['logged_at'] >= start_at)
                                & (events_df['logged_at'] <= end_at)].copy()
    if not selected_df.empty and 'logged_at' in selected_df.columns:
        first_logged_at = selected_df['logged_at'].dropna().min()
        if pd.notna(first_logged_at):
            selected_df['execute_elapsed_seconds'] = (
                selected_df['logged_at'] - first_logged_at).dt.total_seconds()
            selected_df['elapsed_second'] = selected_df[
                'execute_elapsed_seconds'].astype(int)
    return selected_df.reset_index(drop=True), start_at, end_at


buffer_pool_event_log_paths = find_buffer_pool_event_log_paths(
    stats_dbfs_result_dir)
print('Loading buffer pool events from: ' +
      ', '.join(str(path) for path in buffer_pool_event_log_paths))
```

```
raw_buffer_pool_events_df = load_buffer_pool_events(stats_dbfs_result_dir)
buffer_pool_events_df, event_execute_start_at, event_execute_end_at = (
    select_execute_buffer_pool_events(raw_buffer_pool_events_df,
                                      stats_dbfs_result_dir))

if buffer_pool_events_df.empty:
    print('No buffer_pool_event rows found. Run dbfs with '
          'DBFS_BUFFER_POOL_EVENT_LOG=1 and DBFS_STORAGE_LOG_LEVEL=info.')
else:
    if event_execute_start_at is not None and event_execute_end_at is not None:
        print(f'Using BenchBase execute interval: {event_execute_start_at} to '
              f'{event_execute_end_at}.')
    display(buffer_pool_events_df[[
        'event_id',
        'logged_at',
        'execute_elapsed_seconds',
        'type',
        'page_id',
        'frame_id',
        'page_kind',
        'dirty',
        'hit',
        'file',
    ]].tail(10))

    display(
        buffer_pool_events_df.groupby(
            ['type', 'page_kind', 'dirty', 'hit'],
            dropna=False).size().reset_index(name='events').sort_values(
                'events', ascending=False))

    pin_events_df = buffer_pool_events_df[buffer_pool_events_df['type'] ==
                                          'pin'].copy()
    if pin_events_df.empty:
        print('No pin events found. Re-run dbfs after the pin-event logging '
              'change to visualize pin requests and hit-page distribution.')
    else:
        pin_events_per_second_df = (pin_events_df.groupby(
            ['elapsed_second', 'hit']).size().reset_index(name='events'))
        pin_events_per_second_pivot_df = pin_events_per_second_df.pivot(
            index='elapsed_second', columns='hit', values='events').fillna(0)
        pin_events_per_second_pivot_df = pin_events_per_second_pivot_df.rename(
            columns={
                0: 'misses',
                1: 'hits'
            })
        for column in ['hits', 'misses']:
```

```
        if column not in pin_events_per_second_pivot_df.columns:
            pin_events_per_second_pivot_df[column] = 0
    pin_events_per_second_pivot_df['pin requests'] = (
        pin_events_per_second_pivot_df['hits'] +
        pin_events_per_second_pivot_df['misses'])
    pin_events_per_second_pivot_df['hit rate (%)'] = (
        pin_events_per_second_pivot_df['hits'] /
        pin_events_per_second_pivot_df['pin requests'] *
        100).where(pin_events_per_second_pivot_df['pin requests'] > 0)

    fig, axes = plt.subplots(1, 2, figsize=(16, 5))
    ax_left = axes[0]
    ax_right = ax_left.twinx()
    ax_left.bar(pin_events_per_second_pivot_df.index,
                pin_events_per_second_pivot_df['pin requests'],
                color='lightgray',
                label='pin requests')
    ax_left.plot(pin_events_per_second_pivot_df.index,
                 pin_events_per_second_pivot_df['hits'],
                 color='tab:blue',
                 linewidth=1.8,
                 label='hits')
    ax_left.plot(pin_events_per_second_pivot_df.index,
                 pin_events_per_second_pivot_df['misses'],
                 color='tab:orange',
                 linestyle='--',
                 linewidth=1.8,
                 label='misses')
    ax_right.plot(pin_events_per_second_pivot_df.index,
                  pin_events_per_second_pivot_df['hit rate (%)'],
                  color='black',
                  linewidth=1.4,
                  label='hit rate (%)')
    ax_left.set_title('Pin Requests and Hit/Miss per Second')
    ax_left.set_xlabel('Elapsed seconds')
    ax_left.set_ylabel('Calls')
    ax_right.set_ylabel('Hit rate (%)')
    ax_left.grid(True, alpha=0.25)
    lines1, labels1 = ax_left.get_legend_handles_labels()
    lines2, labels2 = ax_right.get_legend_handles_labels()
    ax_left.legend(lines1 + lines2, labels1 + labels2, loc='best')

    top_pin_pages_df = (pin_events_df.assign(
        result=pin_events_df['hit'].map({
            0: 'misses',
            1: 'hits'
        })).groupby(['page_key', 'page_kind',
```

```
                        'result']).size().reset_index(name='events'))
    top_pin_pages_pivot_df = top_pin_pages_df.pivot_table(
        index=['page_key', 'page_kind'],
        columns='result',
        values='events',
        fill_value=0)
    for column in ['hits', 'misses']:
        if column not in top_pin_pages_pivot_df.columns:
            top_pin_pages_pivot_df[column] = 0
    top_pin_pages_pivot_df['pin requests'] = (
        top_pin_pages_pivot_df['hits'] + top_pin_pages_pivot_df['misses'])
    top_pin_pages_pivot_df['hit rate (%)'] = (
        top_pin_pages_pivot_df['hits'] /
        top_pin_pages_pivot_df['pin requests'] *
        100).where(top_pin_pages_pivot_df['pin requests'] > 0)
    top_pin_pages_pivot_df = top_pin_pages_pivot_df.sort_values(
        'pin requests', ascending=False).head(20)
    display(top_pin_pages_pivot_df.reset_index())
    top_pin_pages_pivot_df[['hits', 'misses'
                            ]].plot(kind='barh',
                                    stacked=True,
                                    ax=axes[1],
                                    color=['tab:blue', 'tab:orange'])
    axes[1].set_title('Top Pages by Pin Requests')
    axes[1].set_xlabel('Pin requests')
    axes[1].set_ylabel('Page')
    axes[1].invert_yaxis()

    fig.tight_layout()
    plt.show()

read_evict_events_df = buffer_pool_events_df[
    buffer_pool_events_df['type'].isin(['read', 'evict'])].copy()
if not read_evict_events_df.empty:
    events_per_second_df = (read_evict_events_df.groupby(
        ['elapsed_second', 'type']).size().reset_index(name='events'))
    events_per_second_pivot_df = events_per_second_df.pivot(
        index='elapsed_second', columns='type', values='events').fillna(0)

    fig, axes = plt.subplots(1, 2, figsize=(16, 5))
    event_style_map = {
        'read': {
            'color': 'tab:blue',
            'linestyle': '--',
            'marker': 'o',
            'markersize': 3,
        },
```

```
            'evict': {
                'color': 'tab:orange',
                'linestyle': '-',
                'marker': 'o',
                'markersize': 3,
            },
        }
        for column in events_per_second_pivot_df.columns:
            style = event_style_map.get(column, {})
            axes[0].plot(events_per_second_pivot_df.index,
                         events_per_second_pivot_df[column],
                         label=column,
                         linewidth=1.8,
                         **style)
        axes[0].set_title('Read/Evict Events per Second')
        axes[0].set_xlabel('Elapsed seconds')
        axes[0].set_ylabel('Events')
        axes[0].grid(True, alpha=0.25)
        axes[0].legend(loc='best', frameon=True)

        top_read_evict_pages_df = (read_evict_events_df.groupby(
            ['page_key',
             'type']).size().reset_index(name='events').sort_values(
                'events', ascending=False).head(20))
        top_read_evict_pages_pivot_df = top_read_evict_pages_df.pivot(
            index='page_key', columns='type', values='events').fillna(0)
        top_read_evict_pages_pivot_df = top_read_evict_pages_pivot_df.
↪sort_values(
            list(top_read_evict_pages_pivot_df.columns), ascending=False)
        top_read_evict_pages_pivot_df.plot(kind='barh', ax=axes[1])
        axes[1].set_title('Top Pages by Read/Evict Events')
        axes[1].set_xlabel('Events')
        axes[1].set_ylabel('Page')
        axes[1].invert_yaxis()

        fig.tight_layout()
        plt.show()

        frame_read_evict_events_df = (read_evict_events_df.groupby(
            ['frame_id', 'type']).size().reset_index(name='events'))
        frame_read_evict_events_pivot_df = frame_read_evict_events_df.pivot(
            index='frame_id', columns='type', values='events').fillna(0)
        for column in ['read', 'evict']:
            if column not in frame_read_evict_events_pivot_df.columns:
                frame_read_evict_events_pivot_df[column] = 0
        frame_read_evict_events_pivot_df['total'] = (
            frame_read_evict_events_pivot_df['read'] +
```

```
            frame_read_evict_events_pivot_df['evict'])

        top_read_evict_frames_pivot_df = frame_read_evict_events_pivot_df.
↪sort_values(
            'total', ascending=False).head(20)
        display(top_read_evict_frames_pivot_df.reset_index())

        fig, ax = plt.subplots(figsize=(12, 6))
        frame_read_evict_events_pivot_df['total'].plot(kind='hist',
                                                       bins=50,
                                                       ax=ax,
                                                       color='tab:purple',
                                                       alpha=0.75)
        ax.set_title('Frame Count by Page Replacement Frequency')
        ax.set_xlabel('Page replacements per frame')
        ax.set_ylabel('Frames')
        ax.grid(True, alpha=0.25)
        fig.tight_layout()
        plt.show()
```

```
Loading buffer pool events from: /Users/yohasebe/Documents/dbfs/benchmarking/res
ults/20260619-074745/tpcc/dbfs/dbfs.buffer_pool_events.log
Using BenchBase execute interval: 2026-06-18 22:53:13.606000+00:00 to 2026-06-18
22:54:13.613000+00:00.
```

	event_id	logged_at	execute_elapsed_seconds	\
462750	5444134	2026-06-18 22:54:13.607000+00:00	59.966	
462751	5444135	2026-06-18 22:54:13.607000+00:00	59.966	
462752	5444136	2026-06-18 22:54:13.607000+00:00	59.966	
462753	5444137	2026-06-18 22:54:13.607000+00:00	59.966	
462754	5444138	2026-06-18 22:54:13.607000+00:00	59.966	
462755	5444139	2026-06-18 22:54:13.607000+00:00	59.966	
462756	5444140	2026-06-18 22:54:13.607000+00:00	59.966	
462757	5444141	2026-06-18 22:54:13.607000+00:00	59.966	
462758	5444142	2026-06-18 22:54:13.607000+00:00	59.966	
462759	5444143	2026-06-18 22:54:13.607000+00:00	59.966	

	type	page_id	frame_id	page_kind	dirty	hit	\
462750	pin	346	12063	leaf_index	1	1.000	
462751	pin	346	12063	leaf_index	1	1.000	
462752	pin	4673	11913	heap	1	1.000	
462753	pin	177	9481	internal_index	1	1.000	
462754	pin	1	6933	internal_index	1	1.000	
462755	pin	346	12063	leaf_index	1	1.000	
462756	pin	346	12063	leaf_index	1	1.000	
462757	pin	4673	11913	heap	1	1.000	
462758	pin	177	9481	internal_index	1	1.000	
462759	pin	1	6933	internal_index	1	1.000	

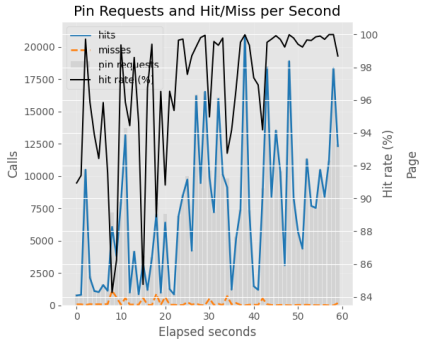
```
                                          file
462750  data/customer.c_w_id__c_d_id__c_id.index
462751  data/customer.c_w_id__c_d_id__c_id.index
462752                          data/customer.db
462753  data/customer.c_w_id__c_d_id__c_id.index
462754  data/customer.c_w_id__c_d_id__c_id.index
462755  data/customer.c_w_id__c_d_id__c_id.index
462756  data/customer.c_w_id__c_d_id__c_id.index
462757                          data/customer.db
462758  data/customer.c_w_id__c_d_id__c_id.index
462759  data/customer.c_w_id__c_d_id__c_id.index
```

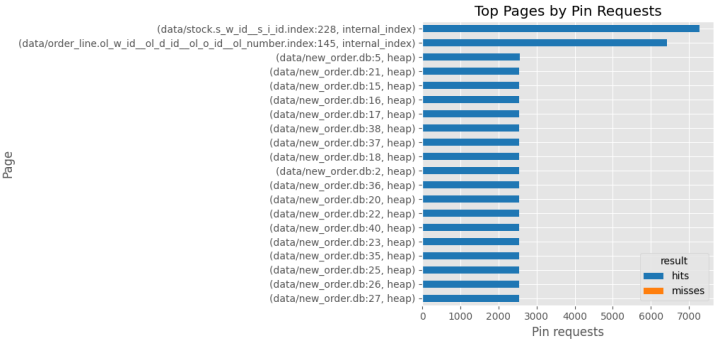
	type	page_kind	dirty	hit	events
4	pin	heap	0	1.000	207478
5	pin	heap	1	1.000	151242
11	pin	leaf_index	1	1.000	33913
8	pin	internal_index	1	1.000	18034
7	pin	internal_index	0	1.000	16958
10	pin	leaf_index	0	1.000	8496
3	pin	heap	0	0.000	7221
12	read	heap	0	NaN	7221
0	evict	heap	1	NaN	6696
2	evict	leaf_index	1	NaN	2308
9	pin	leaf_index	0	0.000	1573
14	read	leaf_index	0	NaN	1573
6	pin	internal_index	0	0.000	16
13	read	internal_index	0	NaN	16
1	evict	internal_index	1	NaN	15

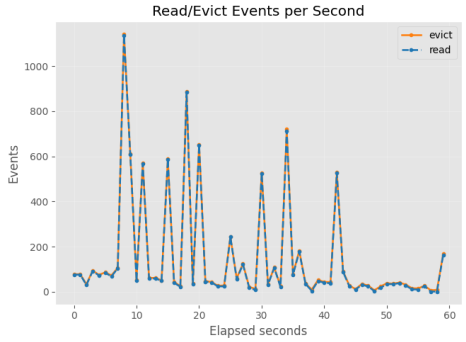
	result	page_key	page_kind	\
0		data/stock.s_w_id__s_i_id.index:228	internal_index	
1	data/order_line.ol_w_id__ol_d_id__ol_o_id__ol_…		internal_index	
2		data/new_order.db:5	heap	
3		data/new_order.db:21	heap	
4		data/new_order.db:15	heap	
5		data/new_order.db:16	heap	
6		data/new_order.db:17	heap	
7		data/new_order.db:38	heap	
8		data/new_order.db:37	heap	
9		data/new_order.db:18	heap	
10		data/new_order.db:2	heap	
11		data/new_order.db:36	heap	
12		data/new_order.db:20	heap	
13		data/new_order.db:22	heap	
14		data/new_order.db:40	heap	
15		data/new_order.db:23	heap	
16		data/new_order.db:35	heap	
17		data/new_order.db:25	heap	

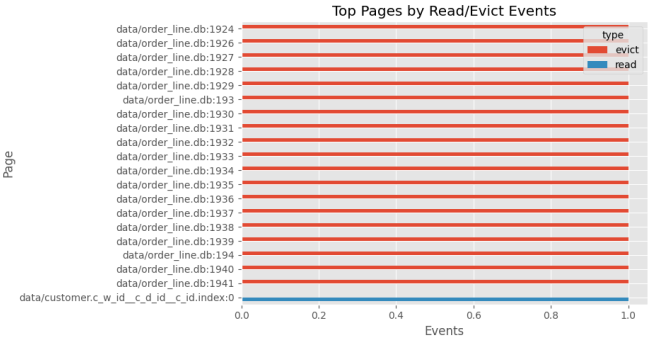
```
18                                  data/new_order.db:26           heap
19                                  data/new_order.db:27           heap
```

result	hits	misses	pin requests	hit rate (%)
0	7,284.000	1.000	7,285.000	99.986
1	6,415.000	0.000	6,415.000	100.000
2	2,550.000	0.000	2,550.000	100.000
3	2,548.000	0.000	2,548.000	100.000
4	2,548.000	0.000	2,548.000	100.000
5	2,548.000	0.000	2,548.000	100.000
6	2,548.000	0.000	2,548.000	100.000
7	2,548.000	0.000	2,548.000	100.000
8	2,548.000	0.000	2,548.000	100.000
9	2,548.000	0.000	2,548.000	100.000
10	2,548.000	0.000	2,548.000	100.000
11	2,548.000	0.000	2,548.000	100.000
12	2,548.000	0.000	2,548.000	100.000
13	2,548.000	0.000	2,548.000	100.000
14	2,548.000	0.000	2,548.000	100.000
15	2,548.000	0.000	2,548.000	100.000
16	2,548.000	0.000	2,548.000	100.000
17	2,548.000	0.000	2,548.000	100.000
18	2,548.000	0.000	2,548.000	100.000
19	2,548.000	0.000	2,548.000	100.000

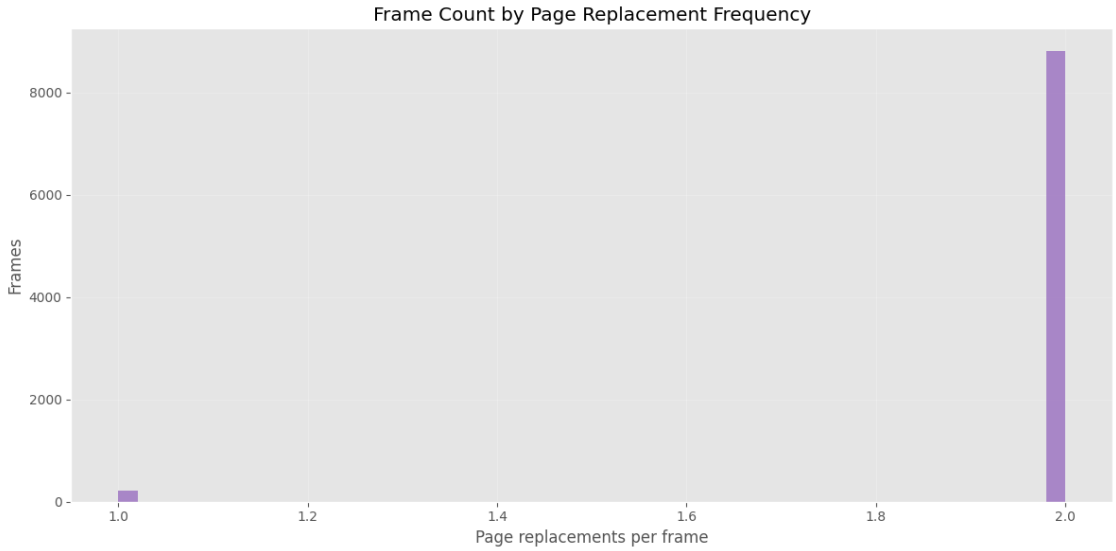








type	frame_id	evict	read	total
0	11549	1.000	1.000	2.000
1	13225	1.000	1.000	2.000
2	13219	1.000	1.000	2.000
3	13220	1.000	1.000	2.000
4	13221	1.000	1.000	2.000
5	13222	1.000	1.000	2.000
6	13223	1.000	1.000	2.000
7	13224	1.000	1.000	2.000
8	13226	1.000	1.000	2.000
9	13217	1.000	1.000	2.000
10	13227	1.000	1.000	2.000
11	13228	1.000	1.000	2.000
12	13229	1.000	1.000	2.000
13	13230	1.000	1.000	2.000
14	13231	1.000	1.000	2.000
15	13232	1.000	1.000	2.000
16	13218	1.000	1.000	2.000
17	13214	1.000	1.000	2.000
18	13124	1.000	1.000	2.000
19	13205	1.000	1.000	2.000



1.3 Comparison Notes

Conclusion template:

- PostgreSQL shows a large throughput advantage over dbfs under the same TPCC configura-

tion.

- dbfs completed the same workload shape without unexpected SQL errors, which means the current SQL support is sufficient for this TPCC slice.
- The main next step is performance-oriented investigation: loader behavior, per-procedure latency spikes, and the gap between dbfs and PostgreSQL throughput.
- If `dbfs_query_trace.csv` and `postgres_query_trace.csv` are present in the result directories, the notebook also ranks query shapes by total time, mean latency, and max latency to highlight bottlenecks on each engine.